1) The uploaded material describes a job-seeker website that retrieves quarterly job vacancy data and a SkillsFuture course directory via API, then shows the Top 10 industries with the highest vacancies for the latest quarter. Users can select an industry to view matched SkillsFuture courses, filter courses by price, duration, and language, and click through to the SkillsFuture site. The site also supports user accounts, favoriting courses, and resume upload/edit with AI skill extraction, with performance and browser compatibility requirements specified.

2) My prompt: Based on the uploaded design materials, recommend a technology stack appropriate for an entry-level software engineering team. We are a team of 5 and we are more familiar with python and java.

AI's response:

Recommended stack (Python-first, simplest to deliver)

Frontend

- Next.js (React + TypeScript)
    - Fast UI development, good for dashboards and filtering (FR2–FR5) Requirements-2
- Tailwind CSS
    - Easy styling + enforces consistent CSS naming (kebab-case) (NFR3.1) Requirements-2
- Component library (optional): shadcn/ui or MUI
    - Speeds up tables, chips, filter panels

Backend (API)

- FastAPI (Python)
    - Very approachable, high performance, clean OpenAPI docs, great for your endpoints (FR1, FR2–FR8)
      Requirements-2
- Pydantic for validation and consistent schemas

Database

- PostgreSQL
    - Store users, favorites, resume metadata, saved analyses (FR7–FR8) Requirements-2
- SQLAlchemy + Alembic (or SQLModel)
    - Migrations are important for teamwork

Caching + background jobs (to meet performance + daily refresh)

- Redis

    - Cache “Top 10 industries” + course matching results so pages load quickly (NFR1.1–NFR1.3)
      Requirements-2
- Celery (Python) + Redis (or APScheduler if you want simpler)
    - Run daily refresh jobs for CSV/Excel ingestion (FR6.1)
      Requirements-2

File storage (resumes)

- S3-compatible storage (AWS S3 / Cloudflare R2 / Supabase Storage)
    - Store uploaded resumes (FR7.1–FR7.2)
      Requirements-2

Auth

- JWT auth (FastAPI users + access tokens) + password hashing (bcrypt/argon2)
    - Supports signup/login and user dashboards/favorites (FR8.1–FR8.4)
      Requirements-2

Resume skill extraction (AI)

Pick one:

- Option A (simple & good): call an LLM API for skill extraction (FR7.4)
  Requirements-2
- Option B (offline baseline): spaCy + keyword dictionary/skills taxonomy, then optionally “upgrade” to LLM later

DevOps / Deployment

- Docker + Docker Compose (frontend, backend, db, redis)
- Deploy: Render / Fly.io (easy) or AWS (more work)
- CI: GitHub Actions (lint/test on PR)

Code conventions (to match your NFR)

- Use snake_case across backend code, DB columns, files (NFR3.2)
  Requirements-2
- Enforce via linters: ruff + black (Python), eslint + prettier (TS)

“Java-friendly” alternative (if you prefer Java backend)

If your team is equally comfortable with Java, swap the backend only:

- Spring Boot (Java) for API + auth

- Keep Next.js + Postgres + Redis + S3 the same
  This is robust, but typically heavier than FastAPI for a university build.

Why this stack fits your requirements

- FR1/FR6: background job pulls vacancy CSV + course Excel daily, stores normalized tables, caches results
  Requirements-2
- FR2–FR5 + NFR1: precompute + cache “Top 10” and course lists; filters can run fast in DB or in-memory cache
  Requirements-2
- FR7: upload resumes to S3, store references in Postgres, run AI extraction async
  Requirements-2
- FR8: Postgres tables for users/favorites/analyses + JWT auth
  Requirements-2

If you share what your “design materials” include (e.g., mockups show server-rendered pages vs SPA, or you already have a UI framework in mind), I can tune this to match that design more closely.

3) FastAPI is appropriate for our project because it is lightweight, straightforward to set up, and matches our team’s stronger familiarity with Python, which reduces the learning curve for an entry-level team. For a university course project with limited time, FastAPI lets us build and document the required API endpoints quickly without the heavier configuration overhead of larger enterprise frameworks, while still being reliable enough for our prototype.